Working Together on This Repo

How we set up git and GitHub, and how we work day to day, so we never lose work or overwrite each other.

The short version

- **One shared repo** on GitHub: `github.com/sashawin7/wnv-climate-california`. No separate copies of the project.
- **Nobody works directly on `main`.** `main` is the version that always works.
- **One branch per task** (not one per person), e.g. `humidity-lags` or `fix-drought-join`. Keep it small and short-lived: a few days, not weeks.
- **Work gets into `main` through a pull request (PR)** that the other person looks over before it is merged.
- **Pull often, push often.** Start every session with `git pull` on `main`; push your branch at the end of every session.

One-time setup

1. Repo owner (Sasha): invite your partner

- On GitHub, open the repo and go to **Settings > Collaborators > Add people**. Enter your partner's GitHub username. They accept the email invite.
- Recommended: protect `main`. Go to **Settings > Branches > Add branch protection rule**, enter `main`, and tick **Require a pull request before merging**. This makes it impossible for either of you to push straight to `main` by accident. (Leave the number of required approvals at 0 or 1; 1 means the other person must click Approve.)

2. Partner: get a copy of the project

Install Git, Python 3.10 or later, and VS Code. Then, in a terminal, in the folder where you keep projects:

```
git clone https://github.com/sashawin7/wnv-climate-california
cd wnv-climate-california
python -m venv .venv
.venv\Scripts\activate            # Mac/Linux: source .venv/bin/activate
pip install -r requirements.txt
```

Data is not stored in git, so download it once by running the pipeline (see the README for timings and options):

```
python -m src.download_wnv
python -m src.download_weather
python -m src.download_gridmet
python -m src.download_drought
python -m src.download_population
python -m src.build_dataset
pytest
```

3. Both of you: tell git who you are (once per computer)

```
git config --global user.name "Your Name"
git config --global user.email "the-email-on-your-github-account"
```

In VS Code, open the project folder, then pick the `.venv` interpreter (Command Palette > *Python: Select Interpreter*).

The daily workflow

Follow these steps every time you start a new piece of work.

Step 1: Start from the latest main

```
git switch main
git pull
```

Step 2: Make a branch for the task

```
git switch -c humidity-lags
```

Name it after *what* you're doing, in lowercase with dashes. Tell your partner what you're picking up (a quick text is fine) so you don't both work on the same file.

Step 3: Work and commit as you go

```
git status                 # what changed?
git add -A                 # stage everything (or: git add src/some_file.py)
git commit -m "Add 2- and 4-week humidity lags to the model"
```

Commit whenever you reach a point that works, even if small. A good message says what changed and, if it isn't obvious, why.

Step 4: Push your branch (at least at the end of each session)

```
git push -u origin humidity-lags    # first push of this branch
git push                            # every push after that
```

Pushing backs up your work and lets your partner see it. It does **not** change `main`.

Step 5: Open a pull request when the task is done

- On GitHub, click **Compare & pull request** (the banner appears after you push), or go to **Pull requests > New**.
- Base: `main`. Compare: your branch. Write 2-3 sentences on what changed and anything you're unsure about.
- Run `pytest` before opening it, so `main` stays working.

Step 6: Review, merge, and clean up

- Your partner reads the **Files changed** tab, leaves comments, and approves.
- If changes are requested, keep committing and pushing to the same branch; the PR updates automatically.
- Click **Merge pull request**, then **Delete branch**.
- Both of you update your local copy:

```
git switch main
git pull
git branch -d humidity-lags         # delete your local copy of the branch
```

Keeping your branch up to date

If your partner merges something into `main` while you're still working, bring their changes into your branch so you're not working on old code:

```
git switch main
git pull
git switch humidity-lags
git merge main
```

Do this before opening a PR, and whenever you know `main` has changed in files you're using.

If you get a merge conflict

A conflict means you both changed the same lines. Git stops and marks them in the file:

```
<<<<<<< HEAD
lags = [1, 2, 4]          # your version
=======
lags = [1, 2, 3, 4, 6]    # the incoming version
>>>>>>> main
```

- In VS Code, click **Accept Current**, **Accept Incoming** or **Accept Both** above each conflict, or edit the text by hand to what it should be. Delete the marker lines.
- Then: `git add -A` and `git commit` (git fills in the message).
- Not sure which version is right? Ask your partner, or ask Claude to explain both sides.

- To give up and go back to before the merge: `git merge --abort`.

Habits that prevent problems

- **Split work by area.** E.g. one person on the download and processing scripts in `src/`, the other on analysis and plots. Two people editing the same file at the same time is what causes conflicts.
- **Notebooks: one owner at a time.** `.ipynb` files merge very badly. Either give each person their own notebook (e.g. `notebooks/sasha_eda.ipynb`) or agree who edits a shared one. Put code you both need into `src/` instead of copying it between notebooks.
- **Don't commit data, the venv or big outputs.** The `.gitignore` already excludes `data/raw`, `data/processed`, `outputs/figures` and `.venv`. Each of you regenerates them by running the pipeline.
- **Record decisions in** `docs/OPEN_QUESTIONS.md` so you both use the same choices (lags, how to treat week 52/53, etc.).
- **New package?** Add it to `requirements.txt` in the same PR, and the other person runs `pip install -r requirements.txt` after pulling.
- **Never force-push** (`git push --force`) to a branch someone else is using, and never to `main`.

Using Claude with this workflow

Claude works best when it can see the shared history. Useful requests:

- "Review my partner's pull request #3 and point out bugs."
- "Explain what changed on the branch `fix-drought-join`."
- "Help me resolve this merge conflict in `src/build_dataset.py`."
- "Commit my changes on a new branch and open a PR."

> **Tip:** Ask Claude to work on a branch, not `main`, and look over what it changed (`git diff` or the PR's *Files changed* tab) before merging, just as you would with your partner's work.

Common situations

Situation	What to do
I committed on `main` by mistake (not pushed yet).	`git switch -c my-new-branch` (your commit comes with you), then `git switch main` and `git reset --hard origin/main`. Ask first if unsure; `reset --hard` discards uncommitted changes.
`git push` says "rejected" / "fetch first".	Someone pushed to that branch first. Run `git pull`, fix any conflicts, then `git push` again.
I need to switch branches but have unfinished changes.	Commit them ("WIP" is fine), or `git stash`, switch, and later `git stash pop`.
I want to throw away my changes to one file.	`git restore path/to/file`
I want to see my partner's branch locally.	`git fetch` then `git switch their-branch`
Pulled, and now the code errors on a missing package.	`pip install -r requirements.txt`

Cheat sheet

Command	What it does
`git status`	Show what's changed and which branch you're on
`git pull`	Get the latest commits from GitHub for this branch
`git switch main`	Go to the main branch
`git switch -c name`	Create a new branch and switch to it
`git add -A`	Stage all changes for the next commit
`git commit -m "msg"`	Save a snapshot locally
`git push -u origin name`	First push of a new branch
`git push`	Send your commits to GitHub
`git merge main`	Bring main's latest changes into your branch
`git log --oneline`	List recent commits
`git diff`	Show changes not yet staged
`git branch -d name`	Delete a merged local branch